Compte rendu

Projet info0406

Maria KINA, Muzhgan OYANI, Roméo TURQUIN, Arnaud LACROIX-FARDET

Table des matières

Table des matières

Cahier des charges : 2
Sujet : 2
Base de données : 3
Ordre de remplissage : 4
Diagrammes de cas d'utilisation : 4
Maquettes du site : 6
Diagramme de navigation : 6
Diagramme de séquence : 8
Code : 9
Laravel : 9
Outils utilisés : 9
Components et vues : 9
CSS et JavaScript : 10
Gestion de la BDD : 10
Android 11
Outils utilisés : 11
Communication avec la base de donnée : 11
Les Activités 11
Design 12
Notifications 13
Améliorations et Fonctionnalités supplémentaires 13
Conclusion : 13

Cahier des charges :

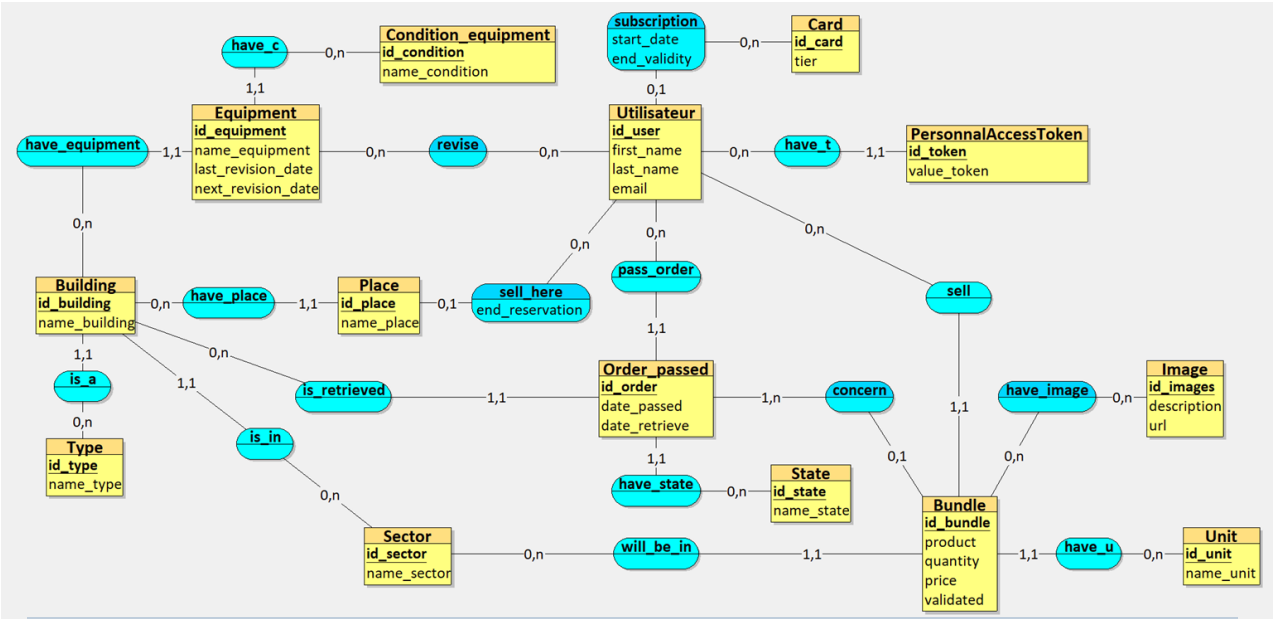
Sujet :

Pour le projet d'INFO0406, il était demandé de développer un site web gérant un marché de gros de type Rungis, ainsi qu'une application mobile chargée de gérer le retrait des commandes.

Ce site doit permettre de gérer les différents bâtiments et secteurs du marché, avec les places disponibles, les équipements adéquats comme les tours à glace pour la conservation des poissons par exemple. Mais également laisser la possibilité pour un marchand de vendre ses produits, et au client d'acheter ces-dits produits. Par conséquent, il faut également fournir une gestion des commandes, avec la possibilité d'en annuler, et d'en conserver une trace.

Base de données :

Pour mettre en place le sujet, il faut une base de données pour permettre la gestion de tous les éléments demandés. Soit le Modèle Conceptuel de Données suivant, suivit du Modèle Logique de Données correspondant :



MLD

Utilisateur = (id_user *INT*, first_name *VARCHAR(50)*, last_name *VARCHAR(50)*, email *VARCHAR(50)*);
State = (id_state *INT*, name_state *VARCHAR(50)*);
Sector = (id_sector *INT*, name_sector *VARCHAR(50)*);
Card = (id_card *INT*, tier *INT*);
Type = (id_type *INT*, name_type *VARCHAR(50)*);
Unit = (id_unit *VARCHAR(50)*, name_unit *VARCHAR(50)*);
Condition_equipment = (id_condition *INT*, name_condition *VARCHAR(50)*);
PersonnalAccessToken = (id_token *INT*, value_token *VARCHAR(50)*, *#id_user*);
Image = (id_images *VARCHAR(50)*, description *VARCHAR(50)*, url *VARCHAR(100)*);
Building = (id_building *INT*, name_building *VARCHAR(50)*, *#id_sector, #id_type*);
Bundle = (id_bundle *INT*, product *VARCHAR(50)*, quantity *INT*, price *CURRENCY*, validated *LOGICAL*, *#id_sector, #id_unit, #id_user*);
Equipment = (id_equipment *INT*, name_equipment *VARCHAR(50)*, last_revision_date *DATE*, next_revision_date *DATE*, *#id_condition, #id_building*);
Place = (id_place *INT*, name_place *VARCHAR(50)*, *#id_building*);
Order_passed = (id_order *INT*, date_passed *DATE*, date_retrieve *DATE*, *#id_building, #id_user, #id_state*);
subscription = (*#id_user*, start_date *VARCHAR(50)*, end_validity *DATE*, *#id_card*);
concern = (*#id_bundle*, *#id_order*);
sell_here = (*#id_place*, end_reservation *DATE*, *#id_user*);
revise = (*#id_user, #id_equipment*);
have_image = (*#id_bundle, #id_images*);

La majorité du MCD reste assez classique et les termes utilisés, que ce soit pour les relations ou les attributs, restent suffisamment explicites pour ne pas avoir besoin de tout expliquer en détail. Cependant il peut être intéressant de noter certains détails.

Premièrement, il y a quatre tables stockant peu d'informations. La table Type permet de spécifier si un bâtiment est un entrepôt, un site de vente, ou des bureaux. La table State permet de définir si une commande est « à récupérer », « annulée », ou autre. La table Unit stocke les termes « kg », « L »,… Et enfin, la table Conditions précise si un équipement est « en réparation », « en maintenance », ou autre.

Ensuite, pour la relation entre les tables Bundles et Orders_passed, la cardinalité est 0,1 – 1,n malgré la présence d'une table pivot dans le MLD. Ce parti pris qui permet de garder une trace de toutes les commandes passées et annulées, mais de n'associer qu'un seul lot (bundle) à une commande payée.

Une autre chose à noter est la table pivot « Revise » entre les tables Utilisateurs et Equipements. Dans le code du projet, cette fonctionnalité n'a pas été implémentée mais la

relation est toujours présente dans le MCD pour laisser la possibilité de gérer du personnel d'entretien et/ou des employés du marché.

Pour reprendre une idée au marché de Rungis, la table Card a été ajoutée. Elle permet de donner les droits liés à Spatie aux utilisateurs d'une part, et d'autre part elle symbolise un abonnement au marché permettant de vendre, d'acheter, ou d'avoir accès au marché pendant une plus courte période (un abonnement de type « Découverte »).

Enfin, les lots de produits ont plusieurs images associées. Cela est dû à l'idée qu'il faut prévoir plusieurs images de différentes tailles pour le côté responsive du site. Ainsi, la table images contient les URLs de différentes images qui pourront ensuite être utilisés dans le code HTML.

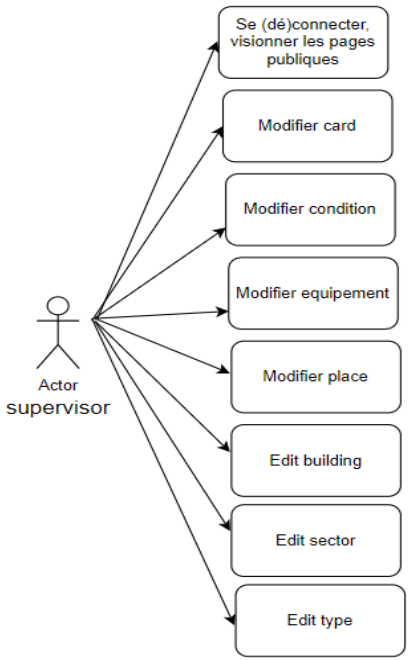
Les tables créées par défaut par Laravel, Spatie,… n'ont pas été ajouté au MCD. Cela comprend notamment les tables de gestion des droits.

Ordre de remplissage :

Les différentes relations entre les tables nécessitent un ordre précis de remplissage pour éviter des erreurs dans la base de données. Tout d'abord, les tables Utilisateur, Type, Card, Unit, State, Sector, et Condition doivent être remplies, car elles sont nécessaires à d'autres tables. Après cela, les bâtiments peuvent être ajoutés, suivis des tables Equipment et Place qui dépendent de la table Building. Ensuite, la table Bundle peut être remplie, et seulement après, la table Order_passed à cause de la cardinalité 1,n.

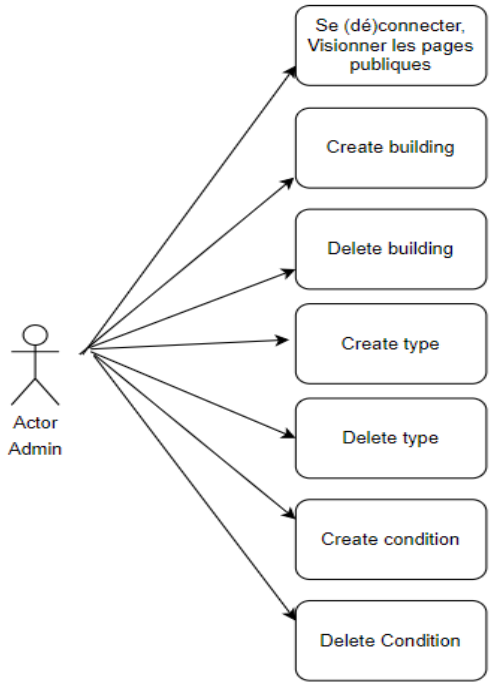
Diagrammes de cas d'utilisation :

Selon le sujet, le site doit pouvoir gérer différents types de droits. Soit les diagrammes suivants :

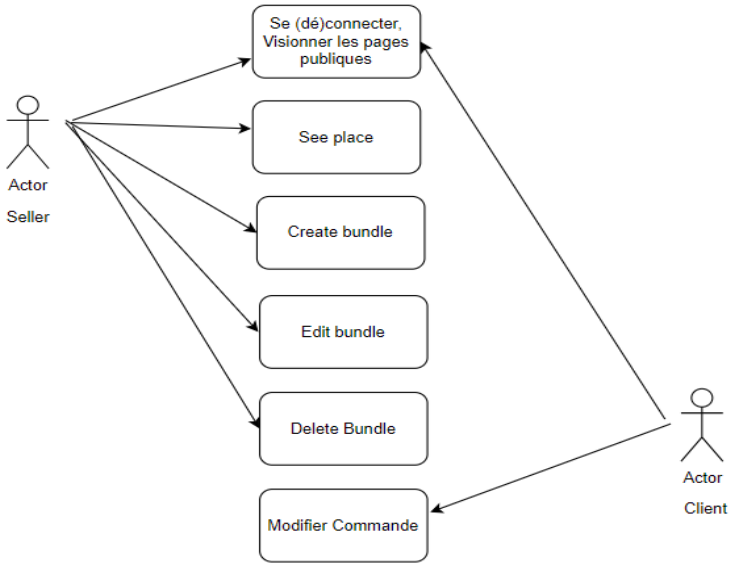


Le superviseur a pour rôle de contrôler les informations présentes sur le site. Ainsi, peut modifier plusieurs éléments liés aux bâtiments, comme leurs types et leurs places, ainsi que les équipements et les conditions associées. Il a aussi la possibilité de gérer les secteurs. Dans le diagramme, « modifier card », « modifier équipement », et « modifier place »

comprennent également la création et la suppression de ces éléments. Ces cas n'ont pas été rajouté pour éviter saturer le diagramme.



L'administrateur possède les même droits qu'un superviseur. En plus de cela, il peut créer et supprimer des bâtiments.



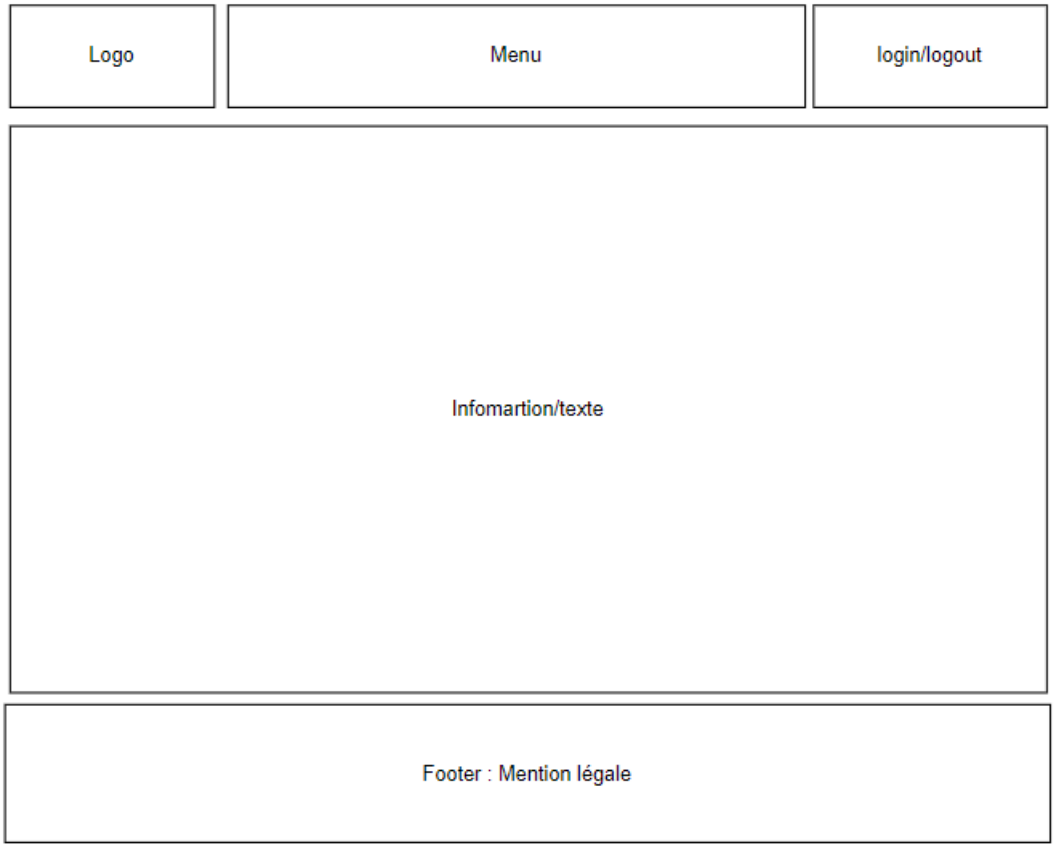
Le marchand peut voir les places disponibles d'un bâtiment et en réserver. Il peut bien évidement gérer ses lots : les renseigner dans le site, les modifier, et les supprimer.

Le client, quant à lui, a la possibilité de commander des produits vendus sur le site et de modifier ses commandes.

Tout rôle peut accéder aux pages publiques, se connecter et se déconnecter.

Maquettes du site :

Pour notre projet voici la maquette qui représente l'ajustement de la majorité des pages de notre site web :

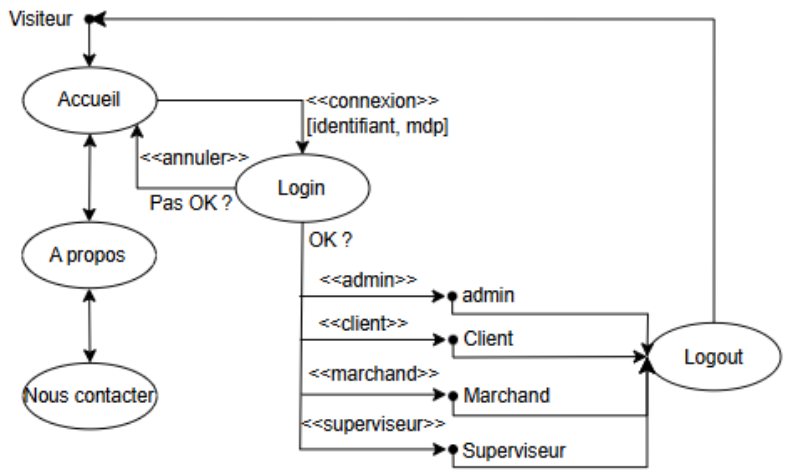


Le header affiche le logo, un menu de navigation et une option pour se connecter ou se déconnecter. Au centre, le contenu change selon la page, avec du texte et des images. Enfin, le footer contient les mentions légales.

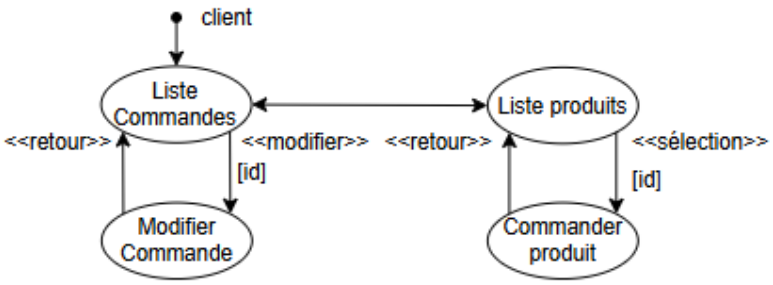
Les seules pages qui ne suivent pas ce schéma sont les pages de formulaire pour la création et l'édition d'éléments. Ces vues ne contiennent pas de header ni de footer.

Diagramme de navigation :

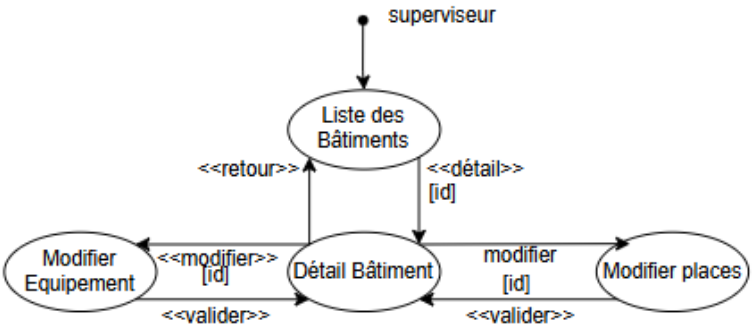
Pour représenter les différents moyens de déplacement sur le site, voici les diagrammes de navigation :



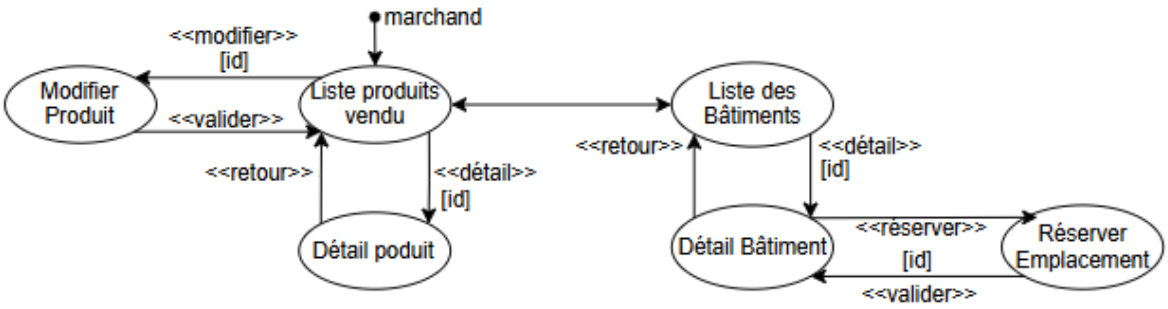
Le premier diagramme présente les pages publiques et la connexion des visiteurs. Un visiteur arrive sur la page d'accueil et peut choisir de se connecter en entrant ses identifiants. Le site vérifie alors son rôle. S'il est reconnu comme administrateur, client, marchand ou superviseur, il est dirigé vers son espace personnel correspondant. S'il n'a pas de rôle particulier, il reste sur la page d'accueil sans accès à d'autres fonctionnalités.



Le deuxième diagramme concerne l'utilisation du site par un client. Une fois connecté, il peut accéder à la liste de ses commandes et peut en modifier certaines si nécessaire. Il a aussi la possibilité de consulter les produits disponibles. Après avoir parcouru le catalogue, il peut sélectionner un article et passer une commande directement depuis son espace.

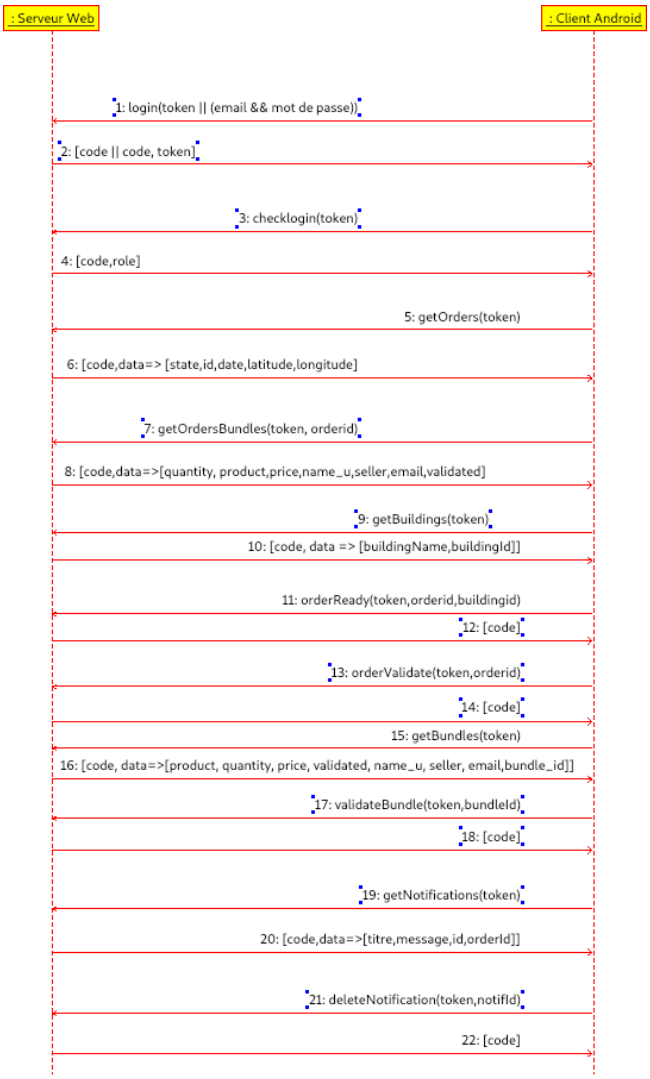


Le troisième diagramme concerne le superviseur. Il a accès à la liste des bâtiments enregistrés sur le site et peut consulter les informations détaillées de chacun. Il peut aussi modifier les équipements installés ou les places disponibles dans ces bâtiments.



Le quatrième diagramme montre les actions possibles pour un marchand. Il peut voir la liste des produits qu'il vend et modifier leurs informations si nécessaire. Il peut aussi accéder à la liste des bâtiments et réserver un emplacement spécifique pour y vendre ses produits.

Diagramme de séquence :



Pour le diagramme de séquences, les différentes opérations ne sont pas dans un ordre prédéfini et cet ordre peut varier en fonction de la visite des activités. Les seules opérations qui sont ordonnées localement sont la connexion qui se fait au début et la récupération des

buildings qui doit se faire avant de valider un bundle. Il est aussi possible de se déconnecter en envoyant le token au serveur web sur la bonne route.

Code :

Laravel :

Outils utilisés :

Pour créer le site, Laravel a été utilisé comme base, avec Breeze pour gérer l'authentification. L'installation s'est faite avec plusieurs commandes : composer create-project --prefer-dist laravel/laravel info0406_web, composer require laravel/breeze --dev, npm update, npm install, php artisan breeze:install et npm run build.

Pour gérer les droits des utilisateurs, Spatie a été ajouté. L'installation s'est faite avec composer require spatie/laravel-permission puis php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider".

La gestion du front-end s'est faite avec Bootstrap et SASS. Ces outils ont été installés avec npm install bootstrap --save-dev, npm install @popperjs/core --save-dev, npm install sass-embedded --save-dev et npm i bootstrap-icons.

Components et vues :

Pour gérer les pages du site, les outils de base fournis lors de l'installation de Laravel ont été utilisé. Les pages héritent donc du template layouts/app.blade.php et la navigation se fait avec le layout/navigation.blade.php. Dans ces deux templates, les components de base ont été utilisé. De plus, un component « display-index » se retrouve dans la majorité des pages pour harmoniser le site. Il est notamment très utilisé pour les pages d'index et de détail, et utilise l'outil Bootstrap Card.

Il y a quelques petites exceptions comme les vues de création et d'édition qui n'ont pas de header ni de footer, ou encore les pages publiques qui n'utilisent pas le component display-index.

La grande majorité des éléments du site (les bâtiments, les lots,…) suivent le même modèle pour afficher et gérer leurs données : il y a une vue pour lister les éléments, une autre pour afficher les détails d'un élément, une pour modifier un élément et une vue pour en créer un nouveau. La suppression d'un élément est gérée directement dans la vue de liste ou de détail grâce à un modal Bootstrap.

Il est à noter que tous les éléments n'ont pas forcément toutes ces vues. Par exemple, les secteurs n'ont pas de vue de détail, car il n'est pas nécessaire d'afficher leurs informations individuellement. Les utilisateurs n'ont pas de vue de création, car leur inscription est déjà prise en charge par Laravel.

Les vues d'édition et de création s'appuient sur des components Blade spécifiques pour gérer les formulaires. De cette façon, les mêmes formulaires peuvent être utilisés sur plusieurs pages sans devoir être recréés à chaque fois. Une fois les données saisies, elles sont envoyées aux Controllers associés.

Note : certaines images, le logo, et le nom du site sont repris directement du site du marché de Rungis. Nous n'avions pas d'imagination concernant notre propre marché.

CSS et JavaScript :

Comme dit précédemment, le front-end du site est géré par Bootstrap et SASS. Ainsi, des classes CSS ont été créé pour harmoniser l'affichage entre les différentes pages, et la couleur « primary » de Bootstrap a été changée pour s'adapter à la couleur du logo.

Concernant le JavaScript, il n'y en a que dans le formulaire de gestion des secteurs pour y associer des bâtiments.

Gestion de la BDD :

Pour gérer les tables de la base de données, la même méthode a été utilisée à chaque fois. Une migration a été créée pour chaque table, parfois plusieurs si des liens entre les tables étaient nécessaires. Ensuite, un modèle, un Seeder, une Factory, un Controller et une Request ont été ajoutés pour chaque table.

Pour les migrations, la commande php artisan make:migration [nom_migration] a été utilisée pour create_buildings_table, create_equipments_table, create_bundles_table, create_orders_table, create_personal_access_token_table, et create_notifications_table. Toutes les autres créations de tables et de tables pivots ont été ajouté dans les migrations déjà existantes : par exemple, les places étant liées aux bâtiments, la création de la table places a été mise dans la migration de create_building_table. Ceci dans le but de réduire le nombre de migrations.

Ensuite, les modèles ont été créés avec php artisan make:model [nom_model]. Les seeders avec php artisan make:seeder [nom_seeder], reprenant les mêmes noms que les modèles, avec en plus un RoleSeeder. Et pour les factories, la commande php artisan make:factory [nom_factory] a été utilisée. Les controllers, quant à eux, ont été créé avec la commande php artisan make:controller [nom_controller]. Enfin, les requests ont été générées avec php artisan make:request [nom_request].

Parti pris :

Chacun des éléments précédents respectent leur rôle habituel, cependant, certaines fonctionnalités demandent des explications.

Tout d'abord, la création et la suppression des secteurs, des unités, des types, des conditions, et des états n'est pas permis car nous avons jugé que les informations liées à ces éléments n'allaient pas changer au cours du temps et allaient être généré lors du tout premier remplissage de la base de données.

Une autre chose à noter est l'ajout de routes pour les Bundles, les Orders, les Places, et les Cards. En effet, certaines fonctionnalités n'étant pas native aux Controllers, elles ont été ajoutées avec des routes spécifiques dans le fichier web.php. Cela concerne la réservation de places, la résignation des abonnements, la majorité de la gestion des commandes (avec l'achat, la résignation,…), et autre.

Concernant l'achat d'une commande, la fonctionnalité n'a pas été implémentée car ce n'est pas le but de ce projet. Ainsi, un simple clic sur un bouton permet de « payer » sa commande. Aussi, un bâtiment est associé à la récupération d'une commande, et donc des coordonnées pour localiser le bâtiment, mais le marché modélisé ici n'existant pas, ces informations sont générées automatiquement.

Pour les cartes, un seul abonnement n'est possible à la fois. De plus, le site est censé vérifier avec un justificatif que la personne qui demande une carte est bien professionnelle. Cette fonctionnalité n'a pas été implémentée car le marché n'existe pas et donc demander un fichier n'aurait pas de sens.

Enfin, lors de la gestion des images, l'utilisation de Chat GPT a été faite pour générer une seule ligne de code (marquée dans le BundleController) pour ne récupérer qu'une image par description identique et transmettre cette sélection aux formulaires de création et d'édition des lots.

Note : La gestion des images ayant été faite très proche de la date de rendu du projet, je (Arnaud) n'avait pas eu le temps de faire suffisamment de recherches pour trouver une alternative à l'IA.

Android

Outils utilisés :

L'IDE android studio a été utilisé afin de développer l'application mobile et une API a été développée afin de communiquer avec la base de données. Des bibliothèques supplémentaires ont aussi été utilisées, pour mettre en place le QR code avec zxing, le rafraîchissement au balayage avec swiperefreshlayout et enfin le worker avec la librairie worker d'android.

Communication avec la base de données :

La communication avec l'API n'a pas été l'idée instinctive qui est venue, puisqu'habituellement il suffit de se connecter directement à la base, mais dans le cadre d'une application Android, cela signifierait que les identifiants de la base soient en claire dans le code de l'application. Cela pose des problèmes de sécurités car l'application peut être décompilée et donc les identifiants récupérés. Il existe des techniques afin de rendre le code java plus opaque (obfuscation), mais il reste un risque que celui-ci puisse être malgré tout déchiffré et compris. Ainsi, une API a été mise en place, qui n'est peut-être pas RESTFUL mais qui est fonctionnelle, elle est sans état, et permet de récupérer les différentes informations nécessaires ainsi que de faire la connexion en assignant un jeton à chaque utilisateur qui se connecte via l'API. Ce jeton est sauvegardé dans le stockage interne, ainsi aucune autre application n'a accès au fichier de ce jeton et en plus depuis Android 10, ces répertoires sont chiffrés, parfait pour sauvegarder des données sensibles.

Les communications avec l'API se font via un thread non bloquant qui va permettre de récupérer les informations demandées et un thread de traitement peut être passé afin de dire comment traiter ces informations. La classe CallAPI est dédiée à cette tâche, elle prend en paramètre de constructeur l'url à qui s'adresse la requête, la méthode utilisée pour celle-ci et enfin les données à envoyer. Le thread de traitement n'a pas été directement inclus dans la classe car celui-ci change d'une activité à l'autre.

Du côté serveur, les requêtes sont vérifiées (vérification des paramètre, de la connexion, les rôles) et s'il y a des problèmes, aucune donné n'est envoyée et un code d'erreur indique le type d'erreur, un code 0 indiquant le succès de l'opération.

Les Activités

Plusieurs activités ont été mises en place afin de répondre aux besoins demandés pour l'application mobile, à savoir, la gestion des commandes et leur retrait. L'activité de lancement qui est FirstActivity, n'a qu'un seul bouton qui permet de se connecter. Cela a été fait en vue d'ajouter la possibilité de créer un compte en plus mais cette fonctionnalité n'a pas été implémentée. Cette activité a une deuxième fonction, elle permet la reconnexion automatiquement si l'utilisateur ne s'est pas déconnecté, puisque qu'elle va récupérer le token sauvegardé localement, et faire une requête pour savoir si la connexion existe toujours.

L'activité suivante dans l'ordre logique est le formulaire de connexion qui fait aussi une requête à l'API avec les identifiants entrés en paramètre. Au départ, il était souhaité d'utiliser les différents contrôleurs existants de Laravel mais un problème se posait : le jeton XSRF qui empêchait justement les requêtes qui ne viennent pas directement de la page. Ainsi, il a fallu faire une connexion à part, mais l'autre problème était le LoginRequest qui empêchait le bon fonctionnement pour les requêtes distantes. Ainsi, il a fallu réécrire cette

partie qui vérifie la connexion et la fonction de hachage a posé quelques problèmes mais finalement la méthode est fonctionnelle.

Après la connexion, on est envoyé sur le tableau de bord, qui affiche les activités possibles en fonction du rôle de l'utilisateur. Les clients peuvent consulter leurs commandes, les vendeurs peuvent consulter leurs lots et les superviseurs peuvent consulter toutes les commandes et changer leur état, les mettre prêtes et les valider. Les trois rôles peuvent bien évidement se déconnecter.

Client

Le client peut donc consulter ses commandes et voir le détail des différents lots dans chaque commande. Il peut aussi afficher le QR code si la commande est prête à être récupérée afin de faciliter le retrait de celle-ci. Dans ce cas-là, le client peut aussi cliquer sur un bouton qui ouvre l'application maps de l'utilisateur et place dessus un point vers le bâtiment où se trouve la commande.

Ici le choix a été de d'ouvrir l'application de carte de l'utilisateur avec un lien geo et de ne pas intégrer la carte directement dans l'application, puisque les solutions proposées sont une intégration de google maps directement. Il n'était pas souhaité d'imposer une solution propriétaire aux utilisateurs et de plus les personnes qui n'ont pas les services googles installés sur leur téléphone ne pourraient pas utiliser la fonctionnalité.

Vendeur

Le vendeur peut consulter ses lots, consulter leur détail et les mettre disponible à la vente.

Superviseur

Le superviseur doit pouvoir consulter toutes les commandes, il peut voir leurs détails et définir une commande comme prête à être récupérée. Dans ce cas, il doit cliquer sur un bouton et choisir le bâtiment dans lequel se trouvera la commande. Il peut aussi valider les commandes pour les retraits, soit avec un numéro de commande qu'il doit renter, soit en scannant le QR code du client avec le scanner de l'application, ou bien avec un scanner différent mais qui n'ouvrira pas directement l'application car seul les filtres communs sont implémentés dans les lecteurs de code et pour faire un lien profond qui s'ouvre dans le navigateur directement, il faut valider le nom de domaine. Ainsi, après avoir scanné avec un scanner externe, il faudra ouvrir le lien dans le navigateur et enfin l'ouvrir dans l'application.

Design

Seul quelques designs ont été faits, pour la connexion, pour le tableau de bord et enfin pour les listview afin qu'elles soient personnalisées (en suivant des tutoriels). Un bon design apporte de l'ergonomie à l'application et l'on s'en rend compte avec les activités sans designs qui sont juste les éléments placés ici et là, qui sont compliqués à prendre en main, il faudrait finaliser cela et utiliser la même direction artistique sur toute l'application, trouver un thème, rajouter une icône et encore de nombreuses choses. Un autre problème d'ergonomie est le manque d'animations et d'indications lorsqu'une action est faite, il faudrait que l'utilisateur soit vraiment conscient de cela, ici un simple toast, mais ce n'est pas suffisant, avec plus de temps, il aurait fallu ouvrir des activités résultat qui indique ce qu'il s'est passé et si tout s'est bien passé. La fluidité de navigation ne se fait pas ressentir, des animations permettraient de corriger cela.

Notifications

Afin de mettre en place les notifications, il a été nécessaire de rajouter une table permettant de stocker les messages liés aux commandes. Des services de notification existent permettant de faire un lien direct avec l'application (des notifications push), mais nativement

c’est un service google qu’il faut utiliser même si firebase n’est pas obligatoire pour envoyer les notifications. D’autre solutions existent mais elles ont un inconvénient, il faut installer une application en plus qui permet de recevoir les notifications et de faire le lien, ce qui n’est pas idéal. Ainsi, il a été choisi d’utiliser un worker, qui permet de faire tourner un job en arrière-plan, et d’exécuter celui-ci à intervalles réguliers. Le seul problème est que l’intervalle minimal est de 15 minutes, mais ici, les notifications ne sont pas une urgence et cette solution a été adoptée. Afin de faire des tests sans attendre le worker, la même fonction présente dans celui-ci a été recopié le code du tableau de bord. Le fonctionnement est simple, lorsqu’une commande est marquée comme étant prête à être récupérée, une entré est ajoutée à la table notification et le worker va périodiquement récupérer les informations des notifications concernant l’utilisateur et les afficher. La notification est supprimée si la commande est validée ou bien si l’utilisateur clique sur la notification.

Améliorations et Fonctionnalités supplémentaires

De nombreuses améliorations sont possibles, tout d’abord, il faudrait faire une vérification sur la connexion internet de l’utilisateur et mettre une activité d’attente si elle n’est pas disponible. L’internalisation serait aussi à faire, c’est facilement faisable mais ce sont de petits détails longs qui n’apporte rien directement, comme le thème, cacher par exemple la barre d’action avec le nom de l’application ou encore ajouter l’icône de l’application et changer le nom de l’application.

La principale fonctionnalité qui serait pratique serait de permettre à un vendeur de créer un produit depuis l’application, puisqu’il pourrait prendre en photo directement son produit simplifiant la création en lui évitant d’importer ses photos vers son ordinateur.

Conclusion :

Une grosse partie de la gestion d’un marché de cette envergure n’a pas été implémentée : la gestion du personnel. Mais comme montré précédemment avec la table pivot entre les tables Utilisateur et Equipment « revise », l’ajout de nouveaux rôles ne serait pas compliqué. Si jamais tous les membres du personnel n’ont pas besoin d’un compte, il faudrait simplement ajouter une table « Personnel », par exemple, dans la base de données.

Également, un point qui a déjà été évoqué, la gestion des images n’est pas du tout optimale. Si on considère tout le cheminement pour ajouter des images, il faudrait que soit un vendeur soit un superviseur ou administrateur ajoute des images via un formulaire, et il faudrait ensuite associer ces images avec les lots correspondant. Mais il y a un problème : si c’est au vendeur d’ajouter ses propres images, la gestion responsive devra être faite par un membre du personnel ayant accès aux modalités des tailles d’images (dans le code actuel, les images font soit 120 soit 150 pixels de large).

Un autre point qui n’a pas été géré par manque de temps est la possibilité de passer des images pour les stocker dans le serveur. Actuellement, il faut le faire manuellement, seul l’association des URL avec des images déjà existantes est possible. Il est a noté qu’il faudrait également gérer la vérification de la taille des images et l’existence des tailles demandées dans la vue bundles/index.blade.php, à savoir 120 et 150 pixels de large.